

VLLM专题（三十六）—自动前缀缓存

AI专题精讲

已于 2025-03-19 10:12:32 修改

阅读量247

收藏

点赞数

CC 4.0 BY-SA版权

分类专栏：

大模型专题系列

文章标签：

缓存



大模型专题系列 专栏收录该内容

您已是超级会员，正在免费阅读会员专享内容

查看更多超级会员权益

PagedAttention 的核心思想是将每个请求的 KV 缓存划分为 KV 块。每个块包含固定数量的标记（tokens）对应的注意力键（keys）和值（values）。PagedAttention 算法允许将这些块存储在非连续的物理内存中，从而通过按需分配内存来消除内存碎片。

为了自动缓存 KV 缓存，我们利用以下关键观察：每个 KV 块可以通过块中的标记以及块之前的前缀标记来唯一标识。

块 1

块 2

块 3

[一阵轻风吹动]

[孩子们的笑声飘过树叶]

[远处传来笑声]

块 1: |<--- 块标记 ---->|

块 2: |<----- 前缀 ----->| |<--- 块标记 ---->|

块 3: |<----- 前缀 ----->| |<--- 块标记 ---->|

在上面的例子中，第一个块的 KV 缓存可以通过标记“一阵轻风吹动”唯一标识。第三个块可以通过块中的标记“远处传来笑声”和前缀标记“一阵轻风吹动孩子们的笑声飘过树叶”唯一标识。因此，我们可以构建以下的一对一映射：

hash(前缀标记 + 块标记) <--> KV 块

通过这个映射，我们可以在 vLLM 的 KV 缓存管理中添加另一个间接层。之前，每个序列在 vLLM 中都维护从其逻辑 KV 块到物理块的映射。为了实现 KV 块的自动缓存，我们将逻辑 KV 块映射到它们的哈希值，并维护一个全局的物理块哈希表。通过这种方式，所有共享相同哈希值的 KV 块（例如，跨两个请求共享的前缀块）可以映射到相同的物理块并共享内存空间。

这个设计实现了自动前缀缓存，而无需在 KV 块之间维护树形结构。更具体地说，所有块都是相互独立的，可以单独分配和释放，这使得我们能够像操作系统中的普通缓存一样管理 KV 缓存。

1. 一般化缓存策略

将所有 KV 块保存在哈希表中，使得 vLLM 可以缓存早期请求的 KV 块，从而节省内存并加速未来请求的计算。例如，如果一个新请求与前一个请求共享系统提示符，那么可以直接使用共享提示符的 KV 缓存，而无需重新计算。然而，KV 缓存的总空间是有限的，因此我们必须在缓存已满时决定保留哪些 KV 块或驱逐哪些 KV 块。

使用哈希表管理 KV 缓存使我们能够实现灵活的缓存策略。例如，在当前的 vLLM 中，我们实现了以下驱逐策略：

- 当没有空闲块时，我们将驱逐引用计数（即当前使用该块的请求数量）为 0 的 KV 块。
- 如果有多个引用计数为 0 的块，我们优先驱逐最久未使用的块（LRU）。
- 如果有多个块的最后访问时间相同，我们优先驱逐位于最长前缀末尾的块（即在它之前有最多块的块）。

注意，当应用于具有完全注意力的模型时，这个驱逐策略实际上实现了与 RadixAttention 中相同的策略，即优先驱逐引用计数为 0 和最久未使用的前缀树中的叶节点。

然而，基于哈希的 KV 缓存管理为我们提供了灵活性，可以处理更复杂的服务场景，并实现更复杂的驱逐策略，超出上述策略：

多 LoRA 适配器服务：在为多个 LoRA 适配器提供服务时，我们可以简单地让每个 KV 块的哈希值也包含请求查询的 LoRA ID，以便为所有适配器启用缓存。通过这种方式，我们可以共同管理不同适配器的 KV 块，从而简化系统实现并提高全局缓存命中率和效率。

多模态模型：当用户输入不仅仅是离散的标记时，我们可以使用不同的哈希方法来处理不同模态输入的缓存。例如，对于图像使用感知哈希来缓存相似的输入图像。



AI专题精讲

已关注

0

0

0

分享

专栏目录

已订阅